

ROOM CALLING SYSTEM

Complete Technical Foundation Document

Internship Project | Netsol IT Solutions Pvt Ltd

Developer: S. Pratham | Stack: Java (Android) + Node.js + WebSocket + SQLite

Version 1.0 | Based on SRS v1.0

1. What Is This App — In Plain Language

This is a hospital nurse call system. In a hospital, patients in different rooms sometimes need help from nurses. Instead of a nurse walking to every room to check, or patients shouting — this app handles that communication digitally and in real-time.

A patient opens a webpage on a device in their room. They press a button — Call, Assistance, Care Required, or Emergency. That signal instantly appears on a monitoring screen at the nurse's station. The nurse sees which room needs what, responds, and resets the room back to idle when done.

That is the entire purpose of this app. Everything else — WebSockets, SQLite, NanoHTTPD, background services — is infrastructure to make that one user flow happen reliably, fast, and without failure.

2. The Three Roles

The same Android APK is installed on every device. On first launch, the app asks: 'What role is this device?' The user selects one. That choice is saved to SharedPreferences and the app always boots directly into that role going forward. A triple-tap gesture on any corner of the screen resets the role selection.

2.1 Server

There is exactly one Server device in the entire system. It is the brain. It owns the database, hosts the web dashboard, and runs the WebSocket hub that every other device connects to. All state changes originate here and are pushed outward.

Think of it as the central post office. Nobody talks to anyone directly — all messages go through the Server.

2.2 Client

A Client is a display screen at a nursing station. It connects to one Server. Its only job is to show the current state of all rooms in a grid and play audio alerts when rooms need attention. It has no write authority — it cannot create, edit, or delete rooms. It can reset a room to Idle (mark it resolved) on behalf of the nurse.

Think of it as a monitoring dashboard on a TV screen at the nurse's desk.

2.3 Master

The Master is a supervisory display, typically used by a head nurse or ward manager. It can connect to multiple Servers simultaneously — one per ward or floor. It rotates between stations every 10 seconds automatically. If any room anywhere hits Emergency (Status 4), the Master shows a full-screen red alert that overrides everything else.

Think of it as the CCTV control room that watches all wards at once.

3. The Four Screens / Interfaces

There are four distinct interfaces in this system. Three are inside the Android app. One is a webpage.

Interface	Type	Who Uses It	Purpose
Role Selection Screen	Android Activity	Anyone setting up the device	Pick Server, Client, or Master on first launch
Server Screen	Android Activity	Admin / IT Staff	Shows IP address and Help URL. Minimal UI.
Client Screen	Android Activity	Nurse at a station	Grid of rooms with live status. Audio alerts.
Master Screen	Android Activity	Head nurse / supervisor	Multi-ward view. Rotates. Emergency overlay.
Web Dashboard (port 8080)	HTML webpage in browser	Admin / nurse	Add, edit, delete rooms. View status.
Patient Room Page (port 8080)	HTML webpage on patient device	Patient	Profile + call buttons. Mark resolved.

4. The Patient Room Page — Detailed

This is the most important user-facing piece and it is NOT part of the Android app UI — it is a webpage served by the Server device's NanoHTTPD on port 8080.

4.1 URL Structure

`http://<server-ip>:8080/room/<RoomID>`

Example: `http://192.168.1.5:8080/room/101`

The admin creates Room 101 from the dashboard. Once created, this URL becomes live. The patient's device (tablet/phone in the room) opens this URL in a browser.

4.2 What the Patient Sees

- Their name (e.g. 'Ramesh Patel')
- Their room number (e.g. 'Room 101')
- Their station (e.g. 'Ward A')
- Five action buttons:
 - Call Nurse (Status 1) — yellow
 - Need Assistance (Status 2) — orange
 - Care Required (Status 3) — red-orange

- EMERGENCY / Code Blue (Status 4) — full red, large
- Mark as Resolved / I'm OK (Status 0) — green
- Current status indicator showing what is currently active for their room

4.3 How It Triggers the Server

When a patient presses a button, the webpage sends an HTTP POST request to the Server:

```
POST http://<server-ip>:8080/update
Body: { "roomId": "101", "status": 1 }
```

The Server receives this, updates SQLite, and immediately broadcasts REFRESH_ALL over WebSocket to all connected Clients and Masters. The entire cycle — button press to screen update — completes in under 1 second on a local network.

Note for Pratham: This is purely HTTP, not WebSocket. The patient page uses simple fetch() or XMLHttpRequest in JavaScript to send the POST. WebSocket is only between the Server and the Android Client/Master apps.

5. System Architecture

5.1 High-Level Data Flow

Every single state change in the system follows this exact path with no exceptions:

1. Patient presses a button on their room webpage
2. Webpage sends HTTP POST to Server (port 8080, NanoHTTPD)
3. Server validates the request
4. Server writes new status to SQLite database
5. Server broadcasts REFRESH_ALL message over WebSocket (port 8090) to all connected Clients and Masters
6. Every Client and Master receives the broadcast
7. Each Client/Master sends GETDETAIL request back to Server to fetch full room list
8. Server responds with full JSON payload of all rooms
9. Client/Master updates its UI grid
10. Audio engine on Client checks highest active priority and plays appropriate sound

The Server never pushes selective updates (e.g. 'only room 101 changed'). It always broadcasts REFRESH_ALL which causes a full re-sync. This is simpler and guarantees no Client ever has stale partial data. The performance cost is acceptable for 10-15 rooms.

5.2 Network Topology

All devices must be on the same local WiFi network. The Server's IP address must be known to all Clients and Masters before they can connect. For demo purposes, use your home router.

Device	Role	Ports It Uses	Direction
Your phone (demo)	Server	8080 (HTTP), 8090 (WebSocket)	Receives connections
Second phone / laptop browser	Patient Room Page	8080 (HTTP)	Sends POST requests to Server
Another phone	Client	8090 (WebSocket)	Connects to Server
Another phone (optional)	Master	8090 (WebSocket)	Connects to Server
Any browser on network	Web Dashboard	8080 (HTTP)	Admin manages rooms

5.3 The 5-Room Demo Scenario

Server is running on your phone at 192.168.1.5. You have 5 rooms: 101, 102, 103, 104, 105. All belong to Ward A. One Client screen is connected. One Master is connected.

- <http://192.168.1.5:8080/room/101> — Patient in Room 101
- <http://192.168.1.5:8080/room/102> — Patient in Room 102
- <http://192.168.1.5:8080/room/103> — Patient in Room 103
- <http://192.168.1.5:8080/room/104> — Patient in Room 104
- <http://192.168.1.5:8080/room/105> — Patient in Room 105
- <http://192.168.1.5:8080/help> — Admin dashboard

Patient in Room 103 presses 'EMERGENCY'. The Client screen immediately shows Room 103 in full red and plays the emergency alarm tone. The Master shows the full-screen Code Blue overlay. The nurse taps 'Resolved' on the Client screen. Room 103 returns to idle. Audio stops.

6. Tech Stack — Every Component Explained

6.1 Android App (Java)

The Android app is written in Java (not Kotlin). You are a beginner in Java/Android, so this document explains every layer.

Component	What It Is	Why It Is Used Here
Activities	An Activity is one screen in Android. Each role (Server, Client, Master) is its own Activity.	Separates concerns. Each role has its own layout XML and Java class.
Services	A Service runs in the background with no visible UI. Android can kill Activities to save memory.	The WebSocket server and NanoHTTPD must keep running even if the screen turns off. A Service survives this.
SharedPreferences	Key-value storage built into Android. Survives app restarts.	Saves the device role (SERVER / CLIENT / MASTER) so the app remembers on reboot.
SQLite	A file-based relational database built into Android. No separate server needed.	Stores all room data: ID, name, station, status, last updated.
NanoHTTPD	A third-party Java library. Creates a tiny HTTP web server inside the Android app.	Serves the web dashboard HTML and patient room pages on port 8080.
Java-WebSocket	A third-party Java library for WebSocket communication.	Runs the WebSocket server on port 8090. Handles real-time broadcast to all Clients and Masters.
OkHttp (Client side)	HTTP client library for Android.	Used by Client/Master to make HTTP requests if needed.
MediaPlayer / RingtoneManager	Built-in Android audio APIs.	Plays notification or alarm sounds when rooms have active statuses.
BroadcastReceiver	Listens for system events like BOOT_COMPLETED.	Auto-starts the Server service after device reboot.
Handler / Looper / Runnable	Android threading primitives.	All UI updates must happen on the main thread. Network code runs on background threads. Handler bridges the two.

6.2 Web Dashboard and Patient Pages (HTML/JS)

These are NOT Android screens. They are plain HTML files bundled inside the Android app (in the assets/ folder) and served by NanoHTTPD when someone opens the URL in a browser.

- Plain HTML, CSS, JavaScript — no framework needed
- Uses fetch() API or XMLHttpRequest for HTTP calls to the Server
- Dashboard uses simple forms to add/edit/delete rooms
- Patient page uses JavaScript button event listeners to POST status updates

6.3 Communication Protocol

Message	Direction	Format	When Sent
GETDETAIL	Client -> Server	Plain text string	After WebSocket connection is established
REFRESH_ALL	Server -> All Clients/Masters	JSON: { action, station, data[] }	After any database update
s[RoomID],[Status]	Server -> Clients (legacy/optional)	Plain text e.g. s101,1	Optional lightweight update
Heartbeat ping	Client -> Server	o (letter o)	Every 15-30 seconds to keep connection alive
POST /update	Patient page -> Server	JSON: { roomId, status }	When patient presses a button
POST /room/add	Dashboard -> Server	JSON: { roomId, roomName, stationName }	Admin adds a room
POST /room/edit	Dashboard -> Server	JSON: { roomId, roomName, stationName }	Admin edits a room
POST /room/delete	Dashboard -> Server	JSON: { roomId }	Admin deletes a room

6.4 Database Schema

```

Table: rooms
  room_id      TEXT  PRIMARY KEY
  room_name    TEXT  NOT NULL
  station_name TEXT  NOT NULL
  status       INTEGER DEFAULT 0    (0=Idle, 1=Call, 2=Assistance, 3=Care,
4=Emergency)
  last_updated TEXT  (ISO 8601 timestamp e.g. 2025-06-08T14:30:00)

```

No separate Patient table is needed at this stage. The patient name can be stored in room_name (e.g. 'Room 101 - Ramesh Patel') or a patient_name column can be added later.

7. Server Module — Deep Dive

7.1 What the Server Screen Shows

The Server's Android screen is intentionally minimal. It shows:

- 'Server Running' status indicator (green dot)

- Local IP address (e.g. 192.168.1.5) — auto-detected
- Dashboard URL: http://192.168.1.5:8080/help
- WebSocket address: ws://192.168.1.5:8090
- Number of connected Clients
- A log area showing recent events (optional but helpful for debugging)

7.2 Startup Sequence

When the Server Activity launches, it starts a foreground Service that does the following in order:

11. Initialize SQLite database (create table if not exists)
12. Start NanoHTTPD on port 8080
13. Start WebSocket Server on port 8090
14. Detect local IP address using NetworkInterface
15. Display IP and Help URL on screen
16. Register BroadcastReceiver for BOOT_COMPLETED (for auto-start on reboot)

7.3 NanoHTTPD Routes

Route	Method	What It Does
GET /help	GET	Serves the web dashboard HTML page
GET /room/:id	GET	Serves the patient room page for that room ID
GET /api/rooms	GET	Returns JSON list of all rooms (used by dashboard)
POST /update	POST	Receives status update from patient page, writes to DB, triggers WebSocket broadcast
POST /room/add	POST	Adds a new room to the database
POST /room/edit	POST	Edits room name or station
POST /room/delete	POST	Deletes a room
POST /room/reset	POST	Resets a room to Idle (Status 0) — callable from nurse Client

7.4 WebSocket Broadcast Logic

Every time the database changes (any POST request that modifies data), the Server immediately does this:

17. Fetch all rooms from SQLite as a JSON array
18. Build the REFRESH_ALL JSON payload:

```
{ "action": "REFRESH_ALL",
  "station": "Ward A",
  "data": [
```



```

        { "room_id": "101", "room_name": "Room 101", "station_name": "Ward A",
          "status": 1, "last_updated": "..."} },
        ...
    ]
}

```

19. Send this JSON string to every active WebSocket connection in the connections list
20. Remove any dead connections (ones that throw an error on send)

7.5 Why a Foreground Service?

Android is aggressive about killing background processes to save battery. If you put WebSocket and NanoHTTPD inside an Activity, Android will kill them the moment the screen turns off or the user switches apps. A foreground Service with a persistent notification (required by Android) tells the OS: 'This process is important, do not kill it.' This is non-negotiable for a server that must run 24/7.

8. Client Module — Deep Dive

8.1 Connection Lifecycle

21. User opens Client Activity and enters the Server IP (e.g. 192.168.1.5)
22. App attempts WebSocket connection to ws://192.168.1.5:8090
23. On successful connection, sends 'GETDETAIL' message
24. Server responds with REFRESH_ALL JSON
25. Client parses JSON and renders room grid
26. Client maintains the connection, sending heartbeat ping every 15-30 seconds
27. If connection drops, Client automatically retries every 5 seconds

8.2 The Room Grid

The Client screen shows all rooms as a grid of cards (RecyclerView with GridLayoutManager). Each card shows:

- Room name / number
- Patient name
- Status label (Idle, Call, Assistance, Care Required, Emergency)
- Color coding:
 - Idle — white/grey
 - Call (1) — yellow
 - Assistance (2) — orange
 - Care Required (3) — deep orange/red
 - Emergency (4) — flashing red
- Last updated timestamp

- A 'Mark Resolved' button visible when status > 0

8.3 Audio Engine

Android provides system ringtone/alarm URIs via RingtoneManager. No external sound files are needed.

Status	Sound Type	Android API
1 — Call	Notification tone	RingtoneManager.TYPE_NOTIFICATION
2 — Assistance	Notification tone (louder)	RingtoneManager.TYPE_NOTIFICATION
3 — Care Required	Alarm tone	RingtoneManager.TYPE_ALARM
4 — Emergency	Alarm tone (max volume, looping)	RingtoneManager.TYPE_ALARM
0 — All Idle	Stop all audio	ringtone.stop()

The audio engine detects the highest active status across all rooms and plays only one sound. If Room 101 is at Status 2 and Room 103 is at Status 4, only the Status 4 alarm plays.

8.4 Reconnection Handling

In the WebSocket onClose() callback, the Client schedules a reconnect attempt using a Handler with a 5-second delay. It retries indefinitely. While disconnected, the screen shows a red 'DISCONNECTED' banner and displays the last known room states with a 'Last sync: [timestamp]' label.

9. Master Module — Deep Dive

9.1 Multi-Server Connection

The Master maintains a list of Server IPs entered by the user. For each IP, it creates a separate WebSocket connection and maintains a separate cache of room data for that station.

9.2 Station Rotation Engine

A Handler runs a timer that fires every 10 seconds, advancing to the next station in the list. The UI updates to show that station's room grid. If the user manually taps a station name to switch to it, the timer resets to 10 seconds from that moment.

9.3 Emergency Overlay

If any incoming REFRESH_ALL payload contains a room with Status 4, regardless of which station it came from, the Master immediately:

28. Stops station rotation
29. Switches to the station with the emergency
30. Overlays a full-screen red panel with flashing text: 'CODE BLUE — [Room Name] — [Station]'
31. Plays the alarm tone at max volume

The overlay auto-dismisses when the emergency room's status returns to 0, or after a configurable timeout (default: 5 minutes). The nurse can also dismiss it manually.

10. Full App Startup Flow

Step	What Happens	Where in Code
1	MainActivity launches	MainActivity.java
2	Check SharedPreferences for saved AppType	MainActivity.java — onCreate()
3a	If AppType exists: launch corresponding Activity directly	startActivity(ServerActivity / ClientActivity / MasterActivity)
3b	If AppType is null: show role selection screen with 3 buttons	activity_main.xml layout
4	User taps role. Save to SharedPreferences. Launch Activity.	MainActivity.java — onClick()
5 (Server)	ServerActivity starts ServerService as foreground service	ServerActivity.java — startForegroundService()
5 (Client)	ClientActivity shows IP entry field, then connects WebSocket	ClientActivity.java
5 (Master)	MasterActivity shows list of Server IPs, connects to each	MasterActivity.java
6 (Triple tap)	Gesture detector clears SharedPreferences, returns to role selection	All Activities — GestureDetector

11. Threading Model — Critical for Beginners

This is the single biggest source of crashes for Android beginners. Read this carefully.

11.1 The Rule

Android has one main thread (also called the UI thread). Every update to any View (TextView, RecyclerView, Button, etc.) MUST happen on the main thread. If you try to update the UI from a background thread, the app crashes immediately with a `CalledFromWrongThreadException`.

11.2 The Problem

WebSocket events (`onMessage`, `onOpen`, `onClose`) arrive on a background thread. SQLite queries should also run on background threads. But you need to update the UI after receiving data.

11.3 The Solution — `runOnUiThread()`

Every Activity has a `runOnUiThread()` method. Use it like this:

```
// In WebSocket onMessage callback (background thread):
websocket.setOnMessageListener(message -> {
    parseRooms(message); // parse JSON — OK on background thread
    runOnUiThread() -> {
        adapter.notifyDataSetChanged(); // update UI — must be on main
        thread
    });
});
```

This is the pattern you will use dozens of times throughout this app. Never update any UI element outside of `runOnUiThread()` when called from a background thread.

12. Error Handling — Every Failure Case

Failure	What the App Does	Where Handled
Client loses WiFi	Show red DISCONNECTED banner. Keep retrying every 5 seconds.	WebSocket <code>onClose()</code> callback
Server goes offline	Client shows last known data + last sync timestamp. Retry continues.	WebSocket <code>onClose()</code> + <code>SharedPreferences</code> cache
Patient page cannot reach Server	Show 'Connection failed' message on the patient webpage (JS fetch error handler).	JavaScript catch block on patient page
SQLite write fails	Log the exception. Show toast to admin on Server screen. Do not crash.	try-catch in DB helper class
Invalid status value received	Ignore the update. Log it.	Input validation in Server route handler

Failure	What the App Does	Where Handled
WebSocket send to dead Client	Catch the exception, remove that Client from the connections list.	Server broadcast loop try-catch
Port already in use (8080/8090)	Show error on Server screen: 'Port 8080 in use. Restart app.'	Server startup try-catch

13. Security

For a demo on a home network, full security is not the priority. But implement these basics:

- Input validation: Room IDs and status values must be validated server-side before writing to SQLite. Reject any status value outside 0-4.
- Local network only: NanoHTTPD and WebSocket only bind to the local network interface. No internet exposure.
- Optional dashboard password: A simple hardcoded PIN check on the /help dashboard route is sufficient for demo.
- SQL injection prevention: Use SQLite's parameterized queries (prepared statements) — never concatenate user input directly into SQL strings.

14. Recommended Android Project Structure

```

app/src/main/
  java/com/yourname/roomcalling/
    MainActivity.java          <- Role selection screen
    ServerActivity.java        <- Server UI screen
    ClientActivity.java        <- Client UI screen (nurse station)
    MasterActivity.java         <- Master UI screen (supervisor)
  service/
    ServerService.java         <- Foreground service running WebSocket +
    NanoHTTPD
  websocket/
    RoomWebSocketServer.java   <- WebSocket server (extends
    WebSocketServer from Java-WebSocket lib)
    RoomWebSocketClient.java   <- WebSocket client (used by Client and
    Master activities)
  http/
    DashboardServer.java       <- NanoHTTPD subclass handling all HTTP
    routes
  db/
    DatabaseHelper.java        <- SQLiteOpenHelper subclass. All DB
    operations here.
  model/
    Room.java                  <- POJO: roomId, roomName, stationName,
    status, lastUpdated

```

```

    adapter/
        RoomAdapter.java          <- RecyclerView adapter for room grid
    util/
        NetworkUtil.java          <- Get local IP address
        AudioManager.java         <- Handles RingtoneManager sound logic
res/
    layout/
        activity_main.xml         <- Role selection
        activity_server.xml
        activity_client.xml
        activity_master.xml
        item_room_card.xml        <- Single room card in the grid
assets/
    dashboard.html               <- Web dashboard (served by NanoHTTTPD)
    room_page.html               <- Patient room page template

```

15. Gradle Dependencies

Add these to your app/build.gradle file under dependencies {}:

```

// WebSocket server and client
implementation 'org.java-websocket:Java-WebSocket:1.5.4'
// Tiny HTTP server (serves dashboard and patient pages)
implementation 'org.nanohttpd:nanohttpd:2.3.1'
// JSON parsing
implementation 'com.google.code.gson:gson:2.10.1'

```

SQLite, SharedPreferences, MediaPlayer, and RecyclerView are all included in the Android SDK or AndroidX. No extra dependencies needed for those.

16. Recommended Build Order

Build this in phases. Do not try to build everything at once.

Phase 1 — Foundation (Week 1)

32. Create Android project in Android Studio (Empty Activity, Java, minSdk 21)
33. Add Gradle dependencies
34. Build MainActivity with role selection (3 buttons, SharedPreferences save)
35. Build DatabaseHelper.java with create/read/update/delete room methods
36. Test SQLite with a simple hardcoded insert and read

Phase 2 — Server HTTP (Week 1-2)

37. Build DashboardServer.java extending NanoHTTPD
38. Serve a hardcoded 'Hello World' response on /help
39. Build the dashboard.html and room_page.html in assets/
40. Wire up POST /update to write to SQLite
41. Test by opening browser on same WiFi network and pressing buttons on patient page

Phase 3 — WebSocket (Week 2)

42. Build RoomWebSocketServer.java
43. On any SQLite change, broadcast REFRESH_ALL JSON to all connections
44. Build RoomWebSocketClient.java for Client Activity
45. Test: open patient page in browser, press button, see WebSocket message received in Client

Phase 4 — Client UI (Week 2-3)

46. Build RecyclerView room grid with RoomAdapter
47. Wire REFRESH_ALL parsing to adapter refresh
48. Add audio engine using RingtoneManager
49. Add reconnection logic

Phase 5 — Master + Polish (Week 3)

50. Build Master Activity with multi-server connection list
51. Add rotation timer (Handler, 10 seconds)
52. Add Emergency overlay
53. Build ServerService as foreground service
54. Add BOOT_COMPLETED auto-start
55. Full end-to-end testing with multiple phones on same WiFi

17. Gaps in the SRS and How We Fill Them

Gap in SRS	Decision Made Here
SRS says 'Room Trigger' but doesn't define how it happens	Patient uses a webpage served by NanoHTTPD at /room/:id with action buttons
SRS doesn't define patient profile	Patient name stored in room record. Page at /room/:id shows name, room, station, buttons.
SRS doesn't specify sound files	Use Android system RingtoneManager tones. No files needed.
SRS doesn't specify who resets to Idle	Both patient (via 'I'm OK' button on room page) and nurse (via 'Mark Resolved' on Client grid) can reset.

Gap in SRS	Decision Made Here
SRS says 'optional dashboard password' but doesn't specify mechanism	Simple hardcoded PIN check in NanoHTTPD route handler for /help. Pin stored in SharedPreferences.
SRS doesn't specify how Client knows Server IP	Client Activity has an IP entry screen on first launch. IP is saved to SharedPreferences.

18. Glossary of Terms

Term	Meaning
Activity	One screen in an Android app. Has a Java class and an XML layout file.
Service	Background process in Android with no UI. Survives screen-off and app switching.
SharedPreferences	Android's built-in key-value store. Like a tiny settings file.
SQLite	A file-based database built into every Android device. No server setup needed.
WebSocket	A persistent two-way network connection. Unlike HTTP (request-response), WebSocket stays open so the server can push data anytime.
NanoHTTPD	A Java library that lets you create a tiny web server inside an Android app.
REFRESH_ALL	The WebSocket message the Server sends to all Clients/Masters when any data changes.
GETDETAIL	The WebSocket message Clients/Masters send to the Server right after connecting to get the full room list.
Foreground Service	An Android Service with a visible notification. Android does not kill it.
RecyclerView	Android's efficient list/grid component. Used to display the room cards.
SSOT (Single Source of Truth)	One place holds the authoritative data. Everything else is a reflection of it. Here, SQLite on the Server is the SSOT.
Status 0-4	Room states: 0=Idle, 1=Call, 2=Assistance, 3=Care Required, 4=Emergency/Code Blue
Port 8080	The port NanoHTTPD listens on for HTTP requests (dashboard, patient pages).
Port 8090	The port the WebSocket server listens on for Client/Master connections.